Runbook - AI Log Anomaly Detector

A step-by-step guide for someone who has never used this tool before.

You do not need to be a programmer to follow this. Every command is written out in full, and every screen is described. If you get stuck, jump to Troubleshooting.

Last updated: 2026-08-16 | Works on macOS, Linux and Windows

---

1. What this tool is

It reads log files - the diaries that servers, firewalls and applications write - and tells you which lines matter.

It uses two workers with different jobs, and knowing the difference explains almost everything about how the screens look:

	The rules	The AI model
What it is	Fixed checks written in code	A local AI (llama3.1:8b)
What it decides	How serious a finding is	Nothing about severity
What it does	Finds and rates the problems	Explains them in plain English
Can it be wrong?	It is predictable and repeatable	It can be inconsistent

The rules always own severity. The AI only explains. That is the core idea. When you see the two disagree on screen, the rule's answer is the one that stands.

What it is not

- ? It is read-only. It never changes a server, blocks an IP, or fixes anything.
- ? It is not a live monitor. You point it at a log file; it does not watch a system.
- ? It does not upload your logs. Everything runs on your machine.

---

2. Before you start

You need three things.

This tool runs on macOS, Linux and Windows.

1. Python 3.9 or newer

Your computer	Check it	If it's missing
macOS	python3 --version	Already there; or brew install python
Linux	python3 --version	sudo apt install python3
Windows	python --version	python.org/downloads - tick "Add Python to PATH" during install

> Windows users: everywhere this guide says python3, type python instead.

2. Ollama - this runs the AI on your own computer.

Download the installer for your system from https://ollama.com/download> (macOS, Windows and Linux are all supported). On Linux you can instead run:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Then download the model - about 5 GB, once - and check it worked:

```
ollama pull llama3.1:8b
ollama list
```

Ollama normally starts by itself after installing. If this tool later says it cannot reach the AI, open a second terminal and run ollama serve.

How much memory do I need? The model needs roughly 8 GB of free RAM. On a 16 GB machine close other big apps first - running low on memory is the most common reason analysis feels slow.

3. The project itself:

```
cd log-analyzer

cp .env.example .env       # macOS / Linux
copy .env.example .env     # Windows (Command Prompt)
```

.env needs no editing for local use - the placeholder API key is deliberate, because a local model does not need a real one.

---

3. Quick start

```
cd log-analyzer
python3 console/serve.py
```

A browser tab opens at http://127.0.0.1:8765/ showing a "Choose a log to analyze" screen. Pick sample-2.log and wait a few seconds.

That is the whole tool. Everything below is detail.

To stop it, press Ctrl-C in the terminal window where it is running. That works on every system and is the recommended way.

If you closed the terminal and it is still running:

System	Command
macOS / Linux	pkill -f console/serve.py
Windows	taskkill /F /IM python.exe (stops all Python), or close the window

---

4. The three ways to give it a log

The picker offers three sources. They are separated on screen because they have genuinely different privacy consequences.

4a. Bundled samples - start here

Buttons for the logs that ship with the project: sample-2.log (19 lines, contains a planted attack) and two real-world 2,000-line logs from the public LogHub dataset.

Best for seeing what the tool does. Nothing leaves your machine.

4b. Your own log file

Click Choose file, pick a .log or .txt from your computer, then Analyze file.

The file is copied to a temporary folder, analyzed, and deleted when you stop the app. It is never uploaded. The screen says so, and it is literally true.

4c. Fetch from a URL

A text box plus one-click chips for public test logs. This is the only source that uses the network, and it works in one direction: it downloads public test data *to* you. Your own logs are never sent anywhere.

That card is styled differently on purpose, so you always know when the network is involved.

5. Compare mode (the "LLM-alone" checkbox)

Below the three sources is a checkbox: Run compare (LLM-alone).

Leave it off for normal use.

When you switch it on, the tool runs the analysis a second time with the rules *removed* - the same AI, the same log, but with no rule findings shown to it - and records what the AI would have called each problem on its own. The console then shows both verdicts side by side:

```
RULE          LLM ALONE     OUTCOME
CRITICAL      HIGH          under-rated by LLM
```

This is how the tool proves its own value rather than asserting it. It doubles the analysis time, which is why it is off by default.

Honest note: measured across six logs, the rules rated a finding higher than the AI alone about 21% of the time, and the AI rated something *higher* than the rules about as often. It is a real effect, not a landslide.

6. Reading the results screen

Top bar

? Findings a raw LLM would have under-rated - only when compare mode ran. If it

did not, it says "compare not run" rather than showing a misleading zero.

? processed locally | 0 bytes leave this machine - a standing statement of fact.
? Run id, time window, hosts, lines parsed - what was analyzed.
? Runs (N) - your saved past runs (see §9).
? New analysis - back to the picker.
? Run manifest | integrity - see §10.

Left: the findings list

Each row shows severity, where it came from, the headline, and a Rule-vs-AI strip.

Filters at the top: All, Rule != LLM, LLM-surfaced, Unreviewed.

Keyboard: j / k move, x select, e mark reviewed.

Right: the detail of one finding

? Rule verdict | authoritative - the severity, and which rule fired.
? LLM alone - what the AI said by itself (compare mode only).
? Outcome - plain language: "Rule overrode the model", "Rule and model agree",

"Model surfaced this".

? Plain-language explanation - the AI's write-up.
? Evidence - the actual log lines, verbatim, with the matched part highlighted.
? Rule predicate that fired - the exact rule, in its own numbers.
? Event sequence - a timeline of what happened, in order.
? Analyst mark - record your judgement (true positive / false positive). This

changes nothing on any server; it is your note.

What the colours mean

Colour	Severity	Meaning
Red	CRITICAL	Act now - e.g. an account was actually broken into
Orange	HIGH	Investigate today
Gold	MEDIUM	Worth a look
Slate	LOW	Context, not an incident
Grey	INFO	Background

There is deliberately no green anywhere on this scale. Green reads as "safe", and nothing on a severity scale should.

---

7. What the tool looks for

Finding	In plain terms
auth_bruteforce	Someone is guessing passwords repeatedly
auth_bruteforce_success	The guessing worked - treat the account as compromised
suspicious_outbound	One of your machines contacted a known-bad address/port
possible_break_in	An outsider is probing your systems
critical_service_event	A service failed - crash, pool exhausted, host down
disk_pressure	Storage is filling up (80% warn, 90% high)
error_rate_spike	A burst of errors in a short window

---

8. The honest screens (important)

This tool is deliberately built to never look more confident than it is. Five screens exist purely to tell you the truth about a run.

"Log format not recognized - 0 of N lines parsed"

The tool could not read your log format. No rule ran. This is not an all-clear - it means nothing was analyzed at all. The AI is skipped entirely in this case, because guessing about text we cannot read would be noise dressed up as coverage.

What to do: check the file is a plain-text log. Supported today: the canonical timestamp LEVEL host message format, and RFC 3164 syslog (sshd/PAM).

"All clear - 0 anomalies"

Lines were parsed, every rule ran, nothing crossed a threshold. This is real good news, and it looks different from the screen above on purpose.

"This run is partial"

Some chunks of the log got no usable answer from the AI. The rules still ran over every line, so severities are complete; only some explanations are missing. Those findings show UNKNOWN, never "missed" - the tool cannot speak for lines the model never rated.

"compare not run"

You did not tick the compare box. The tool says so instead of showing "0 under-rated", which would falsely suggest the AI agreed with everything.

"Explanation pending"

On a large log, only the most severe findings are explained up front. Click a finding (or the Explain this finding button) and its explanation is generated on the spot, in about 15 seconds. Everything factual - severity, evidence, rule, timeline - is already there and does not depend on the AI.

---

9. Reopening past runs

Analyses are saved on your machine. The Runs (N) button lists them; click one to reopen it.

If you close the browser, restart the app, or reboot, the last run is restored automatically - you do not have to analyze again. The picker also lists past runs under "Reopen a saved run".

History lives in console/.runs/ (last 25 runs) and is never committed to git.

---

10. Proving a run is genuine

Click Run manifest | integrity. You get:

```
sha256(input)     7e8b3dfd9c3293ca...
sha256(detector)  43f0560f2a81d52a...
ruleset           v1
model             llama3.1:8b | temp 0
generated         2026-08-16T14:14:41
```

These are fingerprints you can recompute yourself:

System	Command
macOS	shasum -a 256 sample-2.log
Linux	sha256sum sample-2.log
Windows	certutil -hashfile sample-2.log SHA256

Do the same for anomaly_detector.py to check the detector's fingerprint.

If the numbers match, the report describes that exact file analyzed by that exact detector.

The screen says *"integrity hashes - recomputable, not a signature"*, and that wording is deliberate: nothing here proves who produced the run. A green "signature valid" badge would be a lie, so there isn't one.

---

11. Using it without the browser

Analyze a log

```
python3 log_analyzer.py --input sample-2.log --output report
```

Produces report.json (machine-readable) and report.md (human-readable).

Useful options:

Option	What it does
--compare	Also run the LLM-alone pass (doubles the time)
--lines-per-chunk N	Lines per AI request (default 25)
--deep-scan	Ask the AI about every chunk, not just those with findings
--model NAME	Use a different model

--base-url URL	Point at a different OpenAI-compatible endpoint

Rules only - no AI at all

```
python3 anomaly_detector.py --input sample-2.log --output anomalies
```

Instant, no model needed. This is the deterministic half on its own.

Threat-intelligence enrichment (optional)

Matches flagged IPs against threat-intel indicators and maps them to MITRE ATT&CK techniques.

```
python3 log_analyzer.py --input sample-2.log --output report
python3 threat_intel/export_iocs.py report.json > iocs.txt
python3 threat_intel/threat_detector.py --input iocs.txt \
    --stix-bundle threat_intel/demo_threat_intel.json --output threat_report
```

The first run downloads MITRE's ATT&CK data (~46 MB), once. Add --ips-only to export_iocs.py if you ever send data to a third party - it omits your internal hostnames.

11b. Sharing results with someone who has nothing installed

Click Download standalone in the console, or run:

```
python3 console/export.py --latest -o run.html
```

You get one HTML file. Email it, put it in a ticket, or drop it in chat. The person opening it needs no Python, no Ollama, no server and no internet - they double-click it and get the full results screen: findings, severities, evidence, the rule that fired, the timeline, the integrity manifest, and working filters.

It is a *results* export, so the picker, upload and "explain this finding" controls are not present - there is no server behind it to do that work. A banner at the top says so.

---

12. Troubleshooting

Symptom	Cause	Fix
"cannot reach LLM endpoint"	Ollama is not running	ollama serve in another terminal
"model not listed"	Model not downloaded	ollama pull llama3.1:8b
First analysis is slow	Model loads into memory (~9s)	Normal; later runs are warm
Everything is very slow	Machine is low on memory	Close other apps; 16 GB is tight with a 5 GB model
"Log format not recognized"	Unsupported log format	See §8 - the format needs a parser
"This run is partial"	The AI failed on some chunks	Try --lines-per-chunk 15
Port already in use	Another console is running	It reclaims the port itself; if it refuses, something else holds it - use --port 8766
python3: command not found (Windows)	Windows names it python	Use python instead of python3
cp: command not found (Windows)	That is a Unix command	Use copy .env.example .env

Browser shows an old run	Should not happen	Refresh; state is sent with no-cache
Analysis seems stuck	Large log	Watch the progress line: chunk 4 of 23 | 17% | ~5 min left

13. How long things take

Measured on a MacBook Air M4 (16 GB) with llama3.1:8b:

Task	Time
Rules over a 2,000-line log	Under a second
One AI request	~14 seconds warm, ~23 cold
sample-2.log (19 lines)	Seconds
A 2,000-line log	Findings in ~8 seconds, full run ~4-5 minutes
Same log with --compare	Roughly double

Findings always appear first. The rules are instant; the AI's explanations fill in behind a progress bar. You never wait on the model to see what was found.

14. Known limits - read before trusting it

? Two log formats only. Canonical and RFC 3164 syslog. Anything else reports "format not recognized" rather than guessing.

? The 8B model is the weak link. It struggles with large chunks and sometimes omits an explanation. Severities never depend on it.

? Not validated on production logs. Accuracy is verified against a labeled test corpus (17 cases) and public datasets, not your environment.

? Thresholds are generic. 5 failed logins, 80% disk - sensible defaults, not tuned to your systems.

? Analyst marks are not persisted across a reopened run.

? Explanations generated on demand are not saved back into run history.

15. Privacy and safety

? Log data stays on this machine. The AI runs locally.

? The web console binds to 127.0.0.1 only - nothing on your network can reach it.

? It serves exactly the console, its state, and the analyze endpoint. Nothing else on disk is reachable.

? It is read-only toward your systems: it reads a log file and writes reports.

? Secrets are never committed: .env is gitignored, only .env.example is tracked.

? The only outbound traffic is the optional URL fetch (public test logs) and the one-time MITRE ATT&CK download.

16. Checking the tool still works

```
python3 tests/eval/run_eval.py           # 17/17 expected
python3 threat_intel/test_threat_intel.py
python3 console/test_console.py
```

All three run without network or AI. The first checks detection against a labeled corpus of planted attacks and near-misses; the third checks the console renders every state honestly.

---

17. Glossary

Term	Meaning
Log	A computer's running diary of what it did
Anomaly / finding	Something out of the ordinary worth a look
Severity	How serious - Critical, High, Medium, Low, Info
Rule / detector	Fixed code that finds and rates problems
LLM / model	The local AI that writes explanations
Compare mode	Running the AI without the rules, to see what it would say alone
Chunk	A slice of the log (25 lines) sent to the AI
False positive	A false alarm
Brute force	Guessing passwords repeatedly
C2	"Command and control" - an attacker's remote server
MITRE ATT&CK	A public catalogue of attacker techniques
Integrity hash	A fingerprint proving a file has not changed

---

18. Command reference

```
# App
python3 console/serve.py                              # picker
python3 console/serve.py --input sample-2.log         # analyze immediately
python3 console/serve.py --input mylog.log --compare  # with LLM-alone comparison
python3 console/serve.py --report report.json         # reopen a saved report
python3 console/serve.py --port 8766                  # different port

# Command line
python3 log_analyzer.py --input mylog.log --output report
python3 log_analyzer.py --input mylog.log --output report --compare
python3 log_analyzer.py --input big.log --output report --lines-per-chunk 15
python3 anomaly_detector.py --input mylog.log --output anomalies

# Threat intel
python3 threat_intel/export_iocs.py report.json > iocs.txt
python3 threat_intel/threat_detector.py --input iocs.txt \
    --stix-bundle threat_intel/demo_threat_intel.json --output threat_report

# Tests
python3 tests/eval/run_eval.py
python3 threat_intel/test_threat_intel.py
python3 console/test_console.py

# Share a run with someone who has nothing installed
python3 console/export.py report.json -o run.html     # one self-contained HTML file
python3 console/export.py --latest -o run.html        # ...from the most recent run

# Housekeeping
Ctrl-C                                                # stop the app (any system)
pkill -f console/serve.py                             # macOS / Linux, if detached
ollama serve                                          # start the AI runtime
ollama list                                           # what models are installed
```

For the architecture and build history, see PROJECT_HANDOFF.md. For a non-technical overview, see GUIDE.md. To contribute, see CONTRIBUTING.md.